

Phase 3 — Coolify Backend Deployment

Goal

Deploy the MarQira Pulse backend to your VPS (187.77.136.105) using Coolify:

- PostgreSQL 16 (internal/private — port 5432 NOT publicly exposed)
- Redis 7 (internal/private)
- Laravel API at `api.marqira.com`
- Run migrations
- Create first admin user
- Configure backups

Important: This guide assumes you are new to Coolify. Every step includes exactly what to click, what to type, and what you should see afterward.

Prerequisites

Before starting:

- Coolify is installed and running on your VPS (187.77.136.105)
 - You can access the Coolify web UI (typically at `https://vps.marqira.com` or `http://187.77.136.105:8000`)
 - Your GitHub repo `UsmanNaimed/MarQira-Pulse` is accessible
 - DNS records for `api.marqira.com` are pointing to 187.77.136.105
-

Part 1: Create PostgreSQL Database

Step 1.1: Access Coolify Dashboard

1. Open your web browser
2. Navigate to your Coolify URL (e.g., `https://vps.marqira.com` or `http://187.77.136.105:8000`)
3. Log in with your Coolify credentials
4. **What you should see:** The Coolify dashboard with a menu on the left side

Step 1.2: Create a New Project (if needed)

1. Look at the left sidebar
2. If you already have a project (e.g., “MarQira” or “Production”), skip to Step 1.3
3. If not, click “+ New Project” (or similar button, may be labeled “Projects” → “New”)
4. **Project Name:** Type `MarQira` (or any name you prefer)
5. Click “Create” or “Save”
6. **What you should see:** Your new project appears in the list

Step 1.3: Add PostgreSQL Database Resource

1. Click on your project name (e.g., “MarQira”) from the left sidebar or project list
2. Look for a button or tab labeled “Resources”, “+ New Resource”, or “Add Resource”

3. Click it
4. **What you should see:** A list of resource types (Applications, Databases, Services, etc.)

Step 1.4: Select PostgreSQL

1. Click **“Database”** (or **“+ Database”**)
2. **What you should see:** A list of database types (PostgreSQL, MySQL, Redis, etc.)
3. Click **“PostgreSQL”**
4. **What you should see:** PostgreSQL configuration form

Step 1.5: Configure PostgreSQL

Fill in the form with these **exact values**:

Field	Value	Notes
Name	marqira-postgres	This is the internal Docker service name
PostgreSQL Version	16 or 16-alpine	Choose the latest stable 16.x version
Publicly Accessible	UNCHECK / OFF	CRITICAL: Do NOT expose port 5432 publicly
Database Name	marqira_pulse	Initial database name
Username	marqira_app	Application database user (NOT the superuser)
Password	Generate a strong password	Click “Generate” if available, or use a password manager to create a 32+ character random password. Save this password — you’ll need it for the Laravel .env
Superuser Password	Generate a strong password	This is for the postgres superuser. Save this separately — you’ll need it for backups and admin tasks
Persistent Volume	CHECK / ON	Ensures data survives container restarts
Volume Path	/var/lib/postgresql/data	Default path (should be pre-filled)

CRITICAL CHECKS before proceeding:

- [] “Publicly Accessible” is **OFF** (unchecked)

- [] You have **saved both passwords** somewhere safe
- [] Persistent Volume is **ON** (checked)

1. Click **“Create”** or **“Deploy”** or **“Save”**
2. **What you should see:** Coolify starts deploying PostgreSQL. You’ll see logs scrolling. Wait until the status shows **“Running”** or **“Healthy”** (this may take 1-2 minutes)

Step 1.6: Verify PostgreSQL is Internal Only

1. On the PostgreSQL resource page, look for **“Ports”** or **“Network”** section
2. **What you should see:** Port 5432 should show as **internal only** or **not exposed** to the public
3. You should see something like `5432:5432` or `marqira-postgres:5432` (internal Docker network)
4. You should **NOT** see `187.77.136.105:5432` or `0.0.0.0:5432` (that would mean publicly exposed — if you do, STOP and fix it)

Step 1.7: Note the Internal Connection Details

Coolify will show you the internal connection details. You need these for Laravel:

```
Host:      marqira-postgres (or the full Docker network name, e.g., marqira-postgres.coolify)
Port:      5432
Database:  marqira_pulse
Username:  marqira_app
Password:  [the password you generated in step 1.5]
```

Write these down or keep the Coolify page open — you’ll need them in Part 3.

Part 2: Create Redis Instance

Step 2.1: Add Redis Resource

1. Go back to your project page (click “MarQira” in the left sidebar)
2. Click **“+ New Resource”** or **“Add Resource”**
3. Click **“Database”**
4. **What you should see:** List of database types
5. Click **“Redis”**
6. **What you should see:** Redis configuration form

Step 2.2: Configure Redis

Fill in the form:

Field	Value	Notes
Name	marqira-redis	Internal Docker service name
Redis Version	7 or 7-alpine	Choose latest stable 7.x
Publicly Accessible	UNCHECK / OFF	CRITICAL: Keep Redis internal only
Password	Generate a strong password	Click “Generate” or create a 32+ char random password. Save this password
Persistent Volume	CHECK / ON	For data persistence
Volume Path	/data	Default Redis data path

CRITICAL CHECKS:

- [] “Publicly Accessible” is **OFF**
- [] Password is **saved**
- [] Persistent Volume is **ON**

1. Click “**Create**” or “**Deploy**”
2. **What you should see:** Redis deploys (1-2 minutes), status becomes “**Running**” or “**Healthy**”

Step 2.3: Note Redis Connection Details

```
Host:      marqira-redis (or full Docker name)
Port:      6379
Password: [the password you generated in step 2.2]
```

Write these down — you’ll need them for Laravel.

Part 3: Deploy Laravel API

Step 3.1: Add Application Resource

1. Go back to your project (“MarQira”)
2. Click “+ **New Resource**” or “**Add Resource**”
3. Click “**Application**” (NOT Database)
4. **What you should see:** Application source options (Git Repository, Docker Image, Dockerfile, etc.)

Step 3.2: Connect to GitHub Repository

1. Click “**Git Repository**” or “**Public Git Repository**”
2. **Repository URL:** Paste `https://github.com/UsmanNaimed/MarQira-Pulse`
3. **Branch:** Type `main`
4. **What you should see:** Coolify may ask you to authenticate with GitHub or configure a deploy key

If Coolify asks for GitHub authentication:

- Follow the prompts to connect your GitHub account, OR
- If it offers “Deploy Key” option, copy the provided SSH public key and add it to your GitHub repo’s Deploy Keys (Settings → Deploy keys → Add deploy key, paste the key, **check “Allow write access” if needed for deployments**, save)

1. After connecting, **What you should see:** Repository connected, Coolify detected your repo

Step 3.3: Configure Application Build Settings

Now Coolify will ask you to configure how to build and run the app:

Field	Value	Notes
Application Name	marqira-api	Internal identifier
Build Pack	Dockerfile	We have a Dockerfile in apps/api/Dockerfile
Dockerfile Path	apps/api/Dockerfile	Path relative to repo root
Build Context	apps/api	The directory containing the Dockerfile and source
Port	9000	PHP-FPM port (our Dockerfile exposes 9000)

IMPORTANT: Coolify might also ask about a “Start Command” or “CMD” — leave it empty or default, the Dockerfile already specifies `CMD ["php-fpm"]`.

Step 3.4: Add Nginx Service (Reverse Proxy)

Our Dockerfile runs PHP-FPM, but we need Nginx to serve HTTP requests.

Option A: If Coolify supports multi-container apps or “Services”:

1. Look for an option to add a “Service” or “Additional Container”
2. Add an Nginx container:
 - **Image:** nginx:alpine
 - **Port:** 80 (map this to public)
 - **Volume/Config:** Mount infrastructure/docker/nginx.conf to /etc/nginx/conf.d/default.conf
 - **Depends on:** marqira-api (the PHP-FPM container)

Option B: If Coolify doesn’t easily support multi-container:

We’ll need to adjust the approach. For now, let’s assume Coolify can deploy a Docker Compose or we’ll configure Nginx separately. **Proceed to the next step and we’ll verify.**

Step 3.5: Configure Domain

1. Look for “Domains” or “Domain Settings” in the application configuration
2. **Domain:** Type api.marqira.com
3. **HTTPS/SSL: Enable** (Coolify will use Let’s Encrypt)
4. **What you should see:** A field to enter the domain, and an option to enable SSL/HTTPS

Make sure:

- [] Domain is `api.marqira.com`
- [] HTTPS/SSL is enabled
- [] DNS for `api.marqira.com` points to `187.77.136.105` (check this in your DNS provider before proceeding)

Step 3.6: Set Environment Variables

This is **CRITICAL** — the Laravel app needs these to connect to PostgreSQL, Redis, and run correctly.

1. Look for **“Environment Variables”** or **“Env”** tab/section
2. Click **“Add Variable”** or **“+ New”** for each variable below
3. Enter these **EXACT** variables:

```
# Application
APP_NAME="MarQira Pulse API"
APP_ENV=production
APP_KEY= # Leave blank for now – we'll generate this after first deploy
APP_DEBUG=false
APP_URL=https://api.marqira.com

# Logging
LOG_CHANNEL=stack
LOG_LEVEL=warning

# Database (use the values from Step 1.7)
DB_CONNECTION=pgsql
DB_HOST=marqira-postgres
DB_PORT=5432
DB_DATABASE=marqira_pulse
DB_USERNAME=marqira_app
DB_PASSWORD=[PASTE the marqira_app password from Step 1.5]

# Redis (use the values from Step 2.3)
REDIS_HOST=marqira-redis
REDIS_PASSWORD=[PASTE the Redis password from Step 2.2]
REDIS_PORT=6379

# Cache/Session/Queue
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# MarQira Secret Key (for encrypting site secrets)
# Generate with: php -r "echo base64_encode(random_bytes(32));"
# For now, use this temporary one (REPLACE after first deploy):
MARQIRA_SECRET_KEY=dGVzdGtleXRlc3RrZXl0ZXN0a2V5dGVzdGtleQ==

# Trusted Proxies (Coolify/Traefik internal network)
# Adjust if needed – typically Docker uses 172.x or 10.x
TRUSTED_PROXIES=172.16.0.0/12,10.0.0.0/8

# Allowed IPs for n8n
MARQIRA_ALLOWED_IPS=187.77.136.105
```

CRITICAL CHECKS:

- [] `DB_HOST=marqira-postgres` (NOT an IP address, NOT localhost — it's the Docker service name)
- [] `REDIS_HOST=marqira-redis` (same — Docker service name)

- [] Passwords are pasted correctly (no extra spaces)
 - [] `APP_DEBUG=false` (NOT true in production)
1. Click **“Save”** or **“Update”** to save the environment variables

Step 3.7: Deploy the Application

1. Click **“Deploy”** or **“Build & Deploy”** button
2. **What you should see:** Coolify starts building the Docker image from your Dockerfile
 - It will clone the repo
 - Run `docker build` using `apps/api/Dockerfile`
 - Show build logs scrolling
 - This takes 3-5 minutes (Composer install, etc.)
3. **Wait** until you see **“Build successful”** or similar
4. Coolify will then start the container
5. **What you should see:** Status changes to **“Running”** or **“Healthy”**

If the build fails:

- Read the logs carefully
- Common issues: Dockerfile path wrong, missing dependencies, GitHub connection failed
- Fix the issue and click “Rebuild” or “Redeploy”

Part 4: Generate APP_KEY and Run Migrations

Your Laravel app is now running, but it doesn’t have an `APP_KEY` yet and the database is empty.

Step 4.1: Access the Application Container Shell

1. In Coolify, on the `marqira-api` application page, look for **“Terminal”**, **“Console”**, **“Shell”**, or **“Exec”** button
2. Click it
3. **What you should see:** A terminal/shell inside the running container

If Coolify doesn’t provide a built-in shell:

- You’ll need to SSH into your VPS and run `docker exec -it [container-name] bash`
- To find the container name: `docker ps | grep marqira-api`
- Then: `docker exec -it marqira-api bash` (or the exact container name)

Step 4.2: Generate APP_KEY

Inside the container shell, run:

```
php artisan key:generate --show
```

What you should see: A base64-encoded key like:

```
base64:7KjF9X2...random...characters...==
```

Copy this entire key (including `base64:` prefix).

Step 4.3: Update APP_KEY Environment Variable

1. Exit the shell (type `exit` or close the terminal)
2. Go back to the application's **Environment Variables** section in Coolify
3. Find the `APP_KEY` variable (it should be blank or have a placeholder)
4. **Paste** the full key you just generated
5. Click **"Save"** or **"Update"**
6. **Restart the application** (look for "Restart" button in Coolify)
7. Wait for it to become **"Running"** again

Step 4.4: Run Database Migrations

1. Open the container shell again (Terminal/Exec button)
2. Run:

```
php artisan migrate --force
```

What you should see:

```
Migrating: 2024_01_01_000001_create_organizations_table
Migrated: 2024_01_01_000001_create_organizations_table (XX ms)
Migrating: 2024_01_01_000002_create_users_table
Migrated: 2024_01_01_000002_create_users_table (XX ms)
...
[All 9 migrations run successfully]
```

If you see an error:

- Common issues:
- **"SQLSTATE[08006] Unable to connect"** → `DB_HOST`, `DB_PASSWORD`, or `DB_USERNAME` is wrong in env vars
- **"SQLSTATE[42P01] relation does not exist"** → Ignore if this is the first run; it's checking for migrations table
- **"Access denied"** → `DB_PASSWORD` is incorrect

1. Verify migrations succeeded: The command should exit with no errors

Step 4.5: Create First Admin User

Still in the container shell, run:

```
php artisan marqira:create-admin
```

What you should see: Interactive prompts:

Name:

Type your name (e.g., `Usman Naeem`) and press Enter.

Email:

Type your email (e.g., `admin@marqira.com`) and press Enter.

Password (min 12 characters):

Type a strong password (12+ characters) and press Enter. **The password will NOT be visible as you type** — this is normal.

Confirm Password:

Type the same password again and press Enter.

What you should see:

Admin created: Usman Naeem <admin@marqira.com>

Save your admin email and password — you'll need these to log into the dashboard later.

1. Exit the shell: `exit`

Part 5: Testing and Verification

Test 5.1: Health Check

1. Open your browser
2. Go to: `https://api.marqira.com/api/health`
3. **What you should see:**

```
{
  "status": "ok",
  "service": "MarQira Pulse API",
  "timestamp": "2024-01-01T12:34:56Z"
}
```

If you see an error or “502 Bad Gateway”:

- The app failed to start
- Check Coolify logs for the application
- Common issues: missing APP_KEY, DB connection failed, port mapping wrong

Test 5.2: Verify Database Connection

1. Access the container shell again
2. Run:

```
php artisan tinker
```

1. In the Tinker prompt, run:

```
\App\Models\User::count()
```

What you should see:

```
=> 1
```

(Because you created 1 admin user in Step 4.5)

1. Run:

```
\App\Models\Organization::first()->name
```

What you should see:

```
=> "MarQira"
```

1. Exit Tinker: `exit`
2. Exit the shell: `exit`

Test 5.3: Verify PostgreSQL is NOT Publicly Accessible

CRITICAL SECURITY TEST

From your **local computer** (NOT the VPS), open a terminal and run:

```
nc -zv 187.77.136.105 5432
```

What you MUST see:

```
Connection refused
```

or

```
Connection timed out
```

If you see “Connection succeeded” or the connection opens:

- **STOP IMMEDIATELY** — PostgreSQL is publicly exposed
- Go back to Coolify, find the PostgreSQL resource
- Disable “Publicly Accessible”
- Redeploy
- Test again

Test 5.4: Verify Redis is NOT Publicly Accessible

Same test for Redis:

```
nc -zv 187.77.136.105 6379
```

What you MUST see:

```
Connection refused
```

or

Connection timed out

If the connection succeeds, Redis is exposed — fix it immediately.

Part 6: Configure PostgreSQL Backups

Step 6.1: Enable Coolify Backups for PostgreSQL

1. In Coolify, go to the **PostgreSQL resource** page (`marqira-postgres`)
2. Look for **“Backups”** tab or section
3. Click it
4. **Enable Backups** (toggle or checkbox)
5. Configure:
 - **Frequency:** Daily (or every 12 hours if available)
 - **Retention:** Keep at least 7 daily backups
 - **Backup Location:** If Coolify offers S3/object storage integration, configure it. Otherwise, backups will be stored on the VPS (acceptable for now, but plan to move to S3 later)
6. Click **“Save”** or **“Enable”**

Step 6.2: Test Backup Manually

1. Look for **“Run Backup Now”** or **“Manual Backup”** button
2. Click it
3. **What you should see:** Backup starts, completes after 10-30 seconds
4. Verify the backup appears in the backup list

Step 6.3: Document Backup Restore Procedure

If you ever need to restore from backup:

1. In Coolify, go to PostgreSQL resource → Backups
2. Find the backup you want to restore
3. Click **“Restore”** (or download the backup file)
4. **IMPORTANT:** Restoring will overwrite the current database — only do this during a disaster recovery

For now, just knowing backups are enabled is sufficient.

Part 7: Final Verification Checklist

Run through this checklist:

- [] `https://api.marqira.com/api/health` returns `{"status":"ok"}`
- [] PostgreSQL port 5432 is **NOT** accessible from your local computer (`nc -zv 187.77.136.105 5432` fails)
- [] Redis port 6379 is **NOT** accessible from your local computer (`nc -zv 187.77.136.105 6379` fails)

- [] Database has 9 migrations applied (`php artisan migrate:status` shows all green)
- [] Admin user exists (you can log in with the credentials you created)
- [] PostgreSQL backups are enabled and running daily
- [] All environment variables are set correctly (no exposed secrets in logs or public repos)
- [] `APP_DEBUG=false` in production
- [] SSL/HTTPS is working for `api.marqira.com` (no browser warnings)

Troubleshooting

Issue: “502 Bad Gateway” when accessing `api.marqira.com`

Cause: The Laravel app failed to start, or Nginx can’t reach PHP-FPM.

Fix:

1. Check Coolify logs for the application
2. Look for errors in the startup logs
3. Common causes:
 - Missing `APP_KEY`
 - Database connection failed (wrong `DB_HOST` , `DB_PASSWORD` , or `DB_USERNAME`)
 - Redis connection failed
 - Syntax error in code

Issue: Migrations fail with “Connection refused”

Cause: Laravel can’t connect to PostgreSQL.

Fix:

1. Verify `DB_HOST=marqira-postgres` (the Docker service name, NOT `localhost` or an IP)
2. Verify `DB_PASSWORD` matches the password you set in Step 1.5
3. Verify PostgreSQL is running: Check Coolify, the postgres container should show “Running”
4. Try connecting manually:

```
bash
```

```
php artisan tinker
```

```
DB::connection()->getPdo();
```

If this fails, the env vars are wrong.

Issue: “Class ‘Ramsey\Uuid\Uuid’ not found”

Cause: Composer dependencies weren’t installed during the build.

Fix:

1. Check the Dockerfile — it should run `composer install`
2. Rebuild the application in Coolify
3. Check build logs to ensure Composer ran successfully

Issue: Can connect to PostgreSQL from public internet

CRITICAL SECURITY ISSUE

Fix:

1. Go to PostgreSQL resource in Coolify
2. Disable “Publicly Accessible” or “Public Port”

3. Redeploy/restart
 4. Test again: `nc -zv 187.77.136.105 5432` should fail
-

Next Steps

After Phase 3 is complete:

- **Phase 4:** Enrollment + HMAC authentication + heartbeats
 - **Phase 5:** React dashboard at `app.marqira.com`
 - **Phase 6:** Origin IP detection and verification
 - **Phase 7:** Private plugin updates at `updates.marqira.com`
-

Definition of Done — Phase 3

- [x] PostgreSQL 16 running in Coolify, **internal only** (port 5432 NOT publicly exposed)
- [x] Redis 7 running in Coolify, **internal only**
- [x] Laravel API deployed to `https://api.marqira.com`
- [x] SSL/HTTPS enabled via Let's Encrypt
- [x] All environment variables set (APP_KEY, DB_, REDIS_, MARQIRA_SECRET_KEY, TRUSTED_PROXIES)
- [x] Database migrations run successfully (all 9 migrations applied)
- [x] First admin user created
- [x] Health check endpoint returns 200 OK
- [x] Database connection verified via Tinker
- [x] Public exposure test passed (5432 and 6379 blocked from public internet)
- [x] PostgreSQL backups enabled and tested
- [x] `APP_DEBUG=false` in production
- [x] No secrets committed to Git, all via environment variables

STOP and confirm with me before proceeding to Phase 4.